

DIGITAL RADIO LABORATORY

Assignment Report

GNU Radio Companion — Waveform Generation, Quantization & FFT Analysis

Prepared by: Harsh Kumar

1. Aim

- To generate standard periodic waveforms (sine, cosine, square, triangle and sawtooth) in GNU Radio Companion (GRC) and to study their time-domain and frequency-domain (FFT) characteristics.
- To study the effect of quantization resolution on signal fidelity by quantizing a cosine signal using 4-bit and 16-bit uniform quantizers and comparing the resulting quantization noise.
- To analyse the sum of three cosine signals sampled at 8 kHz, in order to illustrate the Nyquist–Shannon sampling theorem and the phenomenon of aliasing.

2. Software Used

- GNU Radio Companion (GRC) — graphical flow-graph design and simulation environment for signal processing.
- Python (used internally by GRC as the QT GUI backend for the generated flow-graphs).

3. Theory

3.1 Standard Periodic Waveforms

A Signal Source block in GRC can generate several standard periodic waveforms — sine, cosine, square, triangle and sawtooth — each defined by a sample rate, frequency and amplitude. Although all of these waveforms are periodic at the same fundamental frequency, they differ greatly in their harmonic (Fourier series) content. A pure sine or cosine wave contains energy at only a single frequency, producing one sharp spectral line (plus its mirror image) in the frequency domain. A square wave, in contrast, can be expressed as an infinite Fourier series of odd harmonics (fundamental, 3rd, 5th, 7th, ... with amplitude falling as $1/n$), so its spectrum shows a fundamental tone accompanied by a comb of odd-harmonic components. A triangle wave also contains only odd harmonics, but because their amplitudes decay much faster (as $1/n^2$), its harmonic sidelobes die out more quickly than those of a square wave. A sawtooth wave contains both even and odd harmonics with amplitude falling as $1/n$, giving it the richest harmonic spectrum of the five waveforms.

3.2 Quantization

Quantization is the process of mapping a continuous range of amplitude values onto a finite set of discrete levels, as required whenever an analog signal is represented digitally. A uniform quantizer with n bits provides 2^n discrete amplitude levels. The difference between the original continuous signal and its quantized version is called the quantization error, and it manifests in the frequency domain as a raised noise floor (quantization noise) spread across the spectrum. The signal-to-quantization-noise ratio (SQNR) is a standard measure of quantizer performance; increasing the number of bits by one improves the SQNR by approximately 6 dB, so a 16-bit quantizer gives roughly 70 dB better SQNR than a 4-bit quantizer for the same signal.

3.3 Nyquist–Shannon Sampling Theorem

The Nyquist–Shannon sampling theorem states that a continuous-time, band-limited signal $x(t)$ whose highest frequency component is $f_{\max}$ can be completely and unambiguously represented by, and reconstructed from, a sequence of its instantaneous samples, provided the sampling frequency f_s satisfies:

$$f_s > 2 \times f_{\max}$$

The quantity $2 \times f_{\max}$ is called the Nyquist rate, and $f_s/2$ is called the Nyquist frequency. When this condition is satisfied, the original continuous-time signal can theoretically be perfectly reconstructed from its samples using an ideal low-pass (reconstruction) filter with cutoff at $f_s/2$. Sampling in the time domain corresponds, in the frequency domain, to periodic replication of the signal's spectrum at intervals of f_s . If $f_s > 2f_{\max}$, these spectral replicas do not overlap and the original baseband spectrum can be recovered cleanly by low-pass filtering. This theorem underlies all analog-to-digital conversion — for example, audio CDs use $f_s = 44.1$ kHz because the audible range extends to about 20 kHz, giving comfortable margin above the 40 kHz Nyquist rate.

3.4 Aliasing

Aliasing is the distortion that occurs when a signal is sampled at a rate lower than the Nyquist rate ($f_s \leq 2f_{\max}$), i.e. when the signal contains frequency components at or above $f_s/2$. Under these conditions the periodic spectral replicas produced by sampling overlap with one another, and a high-frequency component $f > f_s/2$ becomes indistinguishable from a lower "alias" frequency given by $f_{\text{alias}} = |f - k \cdot f_s|$, where k is the integer that folds f into the range $[0, f_s/2]$. Once this folding happens, the true high-frequency content is irrecoverably lost and corrupts the reconstructed signal with false low-frequency content.

Aliasing can be prevented by:

- Anti-aliasing (low-pass) filtering — passing the analog signal through a low-pass filter with cutoff at $f_s/2$ before sampling, so energy above the Nyquist frequency cannot fold back into the baseband.
- Increasing the sampling rate — choosing f_s comfortably greater than $2 \times f_{\max}$ (oversampling) so the Nyquist frequency exceeds the highest frequency present in the signal.
- Band-limiting the source signal at the design stage, so that $f_{\max}$ is known and controlled and an appropriate f_s can be selected in advance.

4. Source Code (GRC Flow-graphs and Block Parameters)

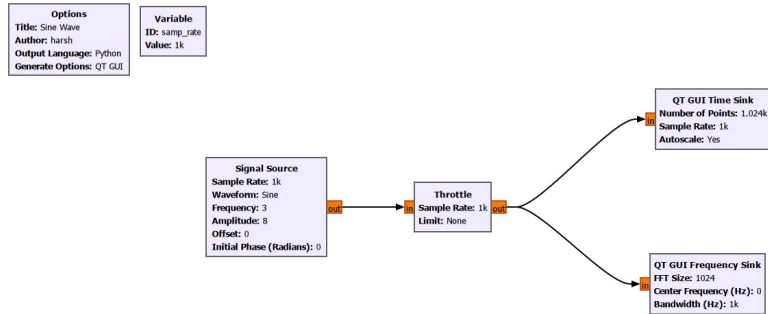
GNU Radio Companion is a graphical (block-diagram based) development environment: instead of hand-written code, the "source" of each experiment is the flow-graph of interconnected blocks and their parameters, shown below. GRC compiles each flow-graph into an underlying Python/GNU Radio script automatically.

4.1 Waveform Generation Flow-graph

A common flow-graph structure was used for all five waveforms: Signal Source → Throttle → { QT GUI Time Sink, QT GUI Frequency Sink }.

Parameter	Value
Sample rate (samp_rate)	1 kHz

Signal frequency	3 Hz
Amplitude	8
Waveform types used	Sine, Cosine, Square, Triangle, Saw Tooth
FFT size (Frequency Sink)	1024

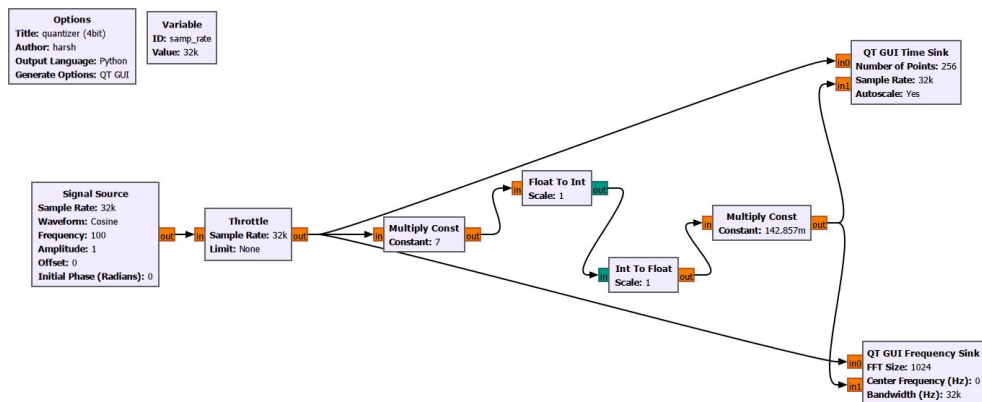


GRC flow-graph — waveform generation (example: Sine)

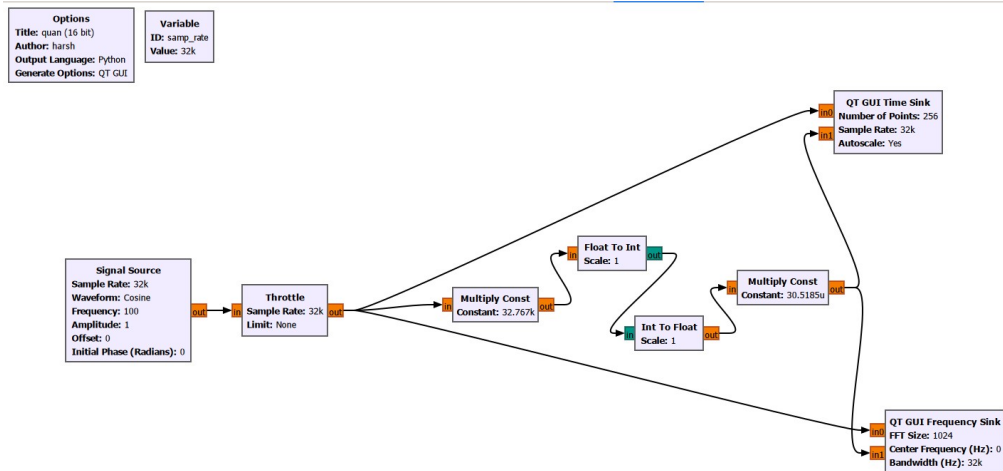
4.2 Quantizer Flow-graph

Quantization was implemented using the block chain: Signal Source (Cosine) → Throttle → Multiply-Const → Float-to-Int → Int-to-Float → Multiply-Const, displayed alongside the original on QT GUI Time Sink / Frequency Sink. The input is scaled up by 2^{n-1} (n = number of bits), rounded to the nearest integer, converted back to floating point, and scaled back down by the reciprocal factor.

Parameter	Value
Sample rate	32 kHz
Signal frequency	100 Hz
Amplitude	1
4-bit scale constant	7 ($2^3 - 1$)
16-bit scale constant	$\approx 32.767k$ ($2^{15} - 1$)



GRC flow-graph — 4-bit quantizer

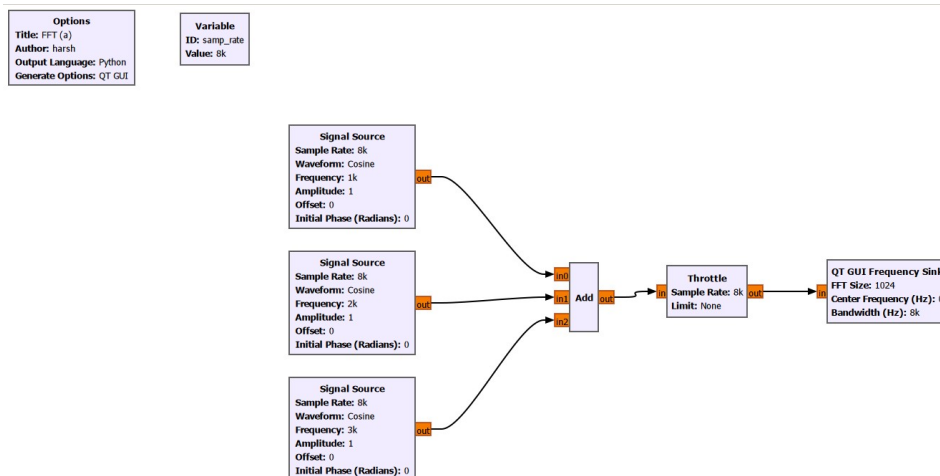


GRC flow-graph — 16-bit quantizer

4.3 FFT / Aliasing Flow-graph

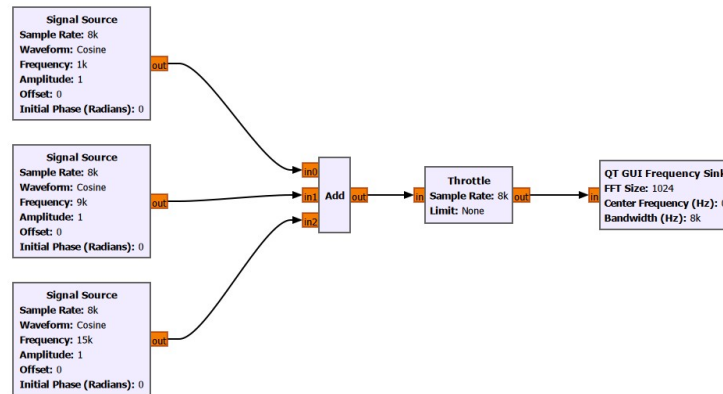
Three cosine Signal Source blocks were summed with an Add block, throttled to real time, and observed on a QT GUI Frequency Sink.

Parameter	Value
Sample rate (fs)	8 kHz
Amplitude (each tone)	1
FFT size	1024
Frequency span shown	± 4 kHz
Case (a) frequencies	f1 = 1 kHz, f2 = 2 kHz, f3 = 3 kHz
Case (b) frequencies	f1 = 1 kHz, f2 = 9 kHz, f3 = 15 kHz



GRC flow-graph — three cosines summed, case (a)

Options	Variable
Title: FFT (b)	ID: samp_rate
Author: harsh	Value: 8k
Output Language: Python	
Generate Options: QT GUI	



GRC flow-graph — three cosines summed, case (b)

5. Procedure

5.1 Waveform Generation

- Open GNU Radio Companion and create a new flow-graph.
- Place a Signal Source block; set sample rate = 1 kHz, frequency = 3 Hz, amplitude = 8.
- Connect the Signal Source to a Throttle block to pace the simulated sample stream to real time.
- Connect the Throttle output to a QT GUI Time Sink (time-domain view) and a QT GUI Frequency Sink (FFT size = 1024) for the frequency-domain view.
- Run the flow-graph and record the Time Sink / Frequency Sink display.
- Change the Signal Source waveform type in turn to Sine, Cosine, Square, Triangle and Saw Tooth, re-running the flow-graph after each change and recording the output.

5.2 Quantization

- Create a Cosine Signal Source (sample rate = 32 kHz, frequency = 100 Hz, amplitude = 1) followed by a Throttle block.
- Build the quantizer chain: Multiply-Const (scale up by 2^{n-1}) → Float-to-Int → Int-to-Float → Multiply-Const (scale down by the reciprocal factor).
- Feed both the original cosine and the quantized signal into a common QT GUI Time Sink and QT GUI Frequency Sink so the two traces can be compared directly.
- Set $n = 4$ (scale constant = 7) and run the flow-graph; record the Time Sink / Frequency Sink output.
- Repeat with $n = 16$ (scale constant $\approx 32.767k$) and record the output.

5.3 FFT of Summed Cosines / Aliasing

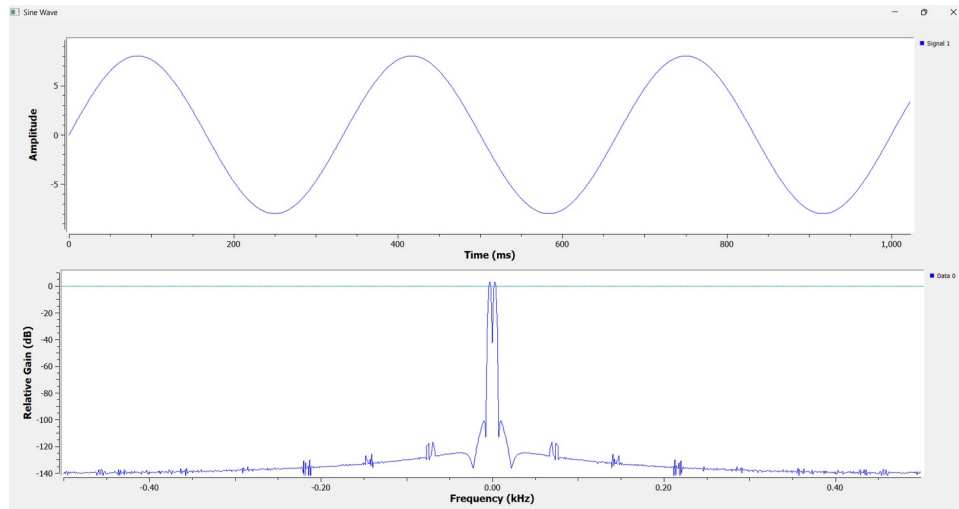
- Create three Cosine Signal Source blocks, each with amplitude = 1 and sample rate $f_s = 8$ kHz.
- Sum the three sources using an Add block, then pass the result through a Throttle block.
- Connect the output to a QT GUI Frequency Sink (FFT size = 1024, span = ± 4 kHz).

- Case (a): set $f_1 = 1$ kHz, $f_2 = 2$ kHz, $f_3 = 3$ kHz; run the flow-graph and record the FFT output.
- Case (b): change the frequencies to $f_1 = 1$ kHz, $f_2 = 9$ kHz, $f_3 = 15$ kHz; run the flow-graph again and record the FFT output.

6. Observation

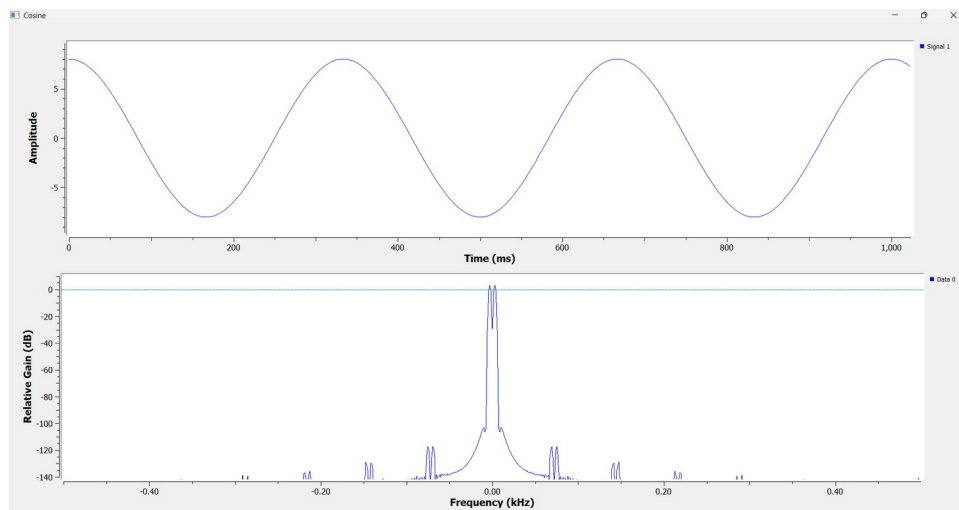
6.1 Waveform Outputs

6.1.1 Sine Wave



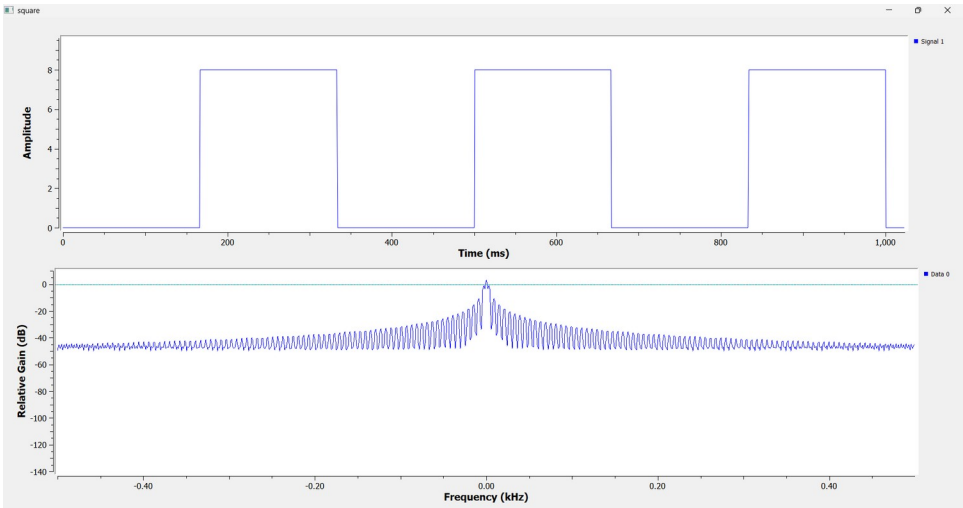
Time-domain (top) and frequency-domain (bottom) output — Sine wave

6.1.2 Cosine Wave



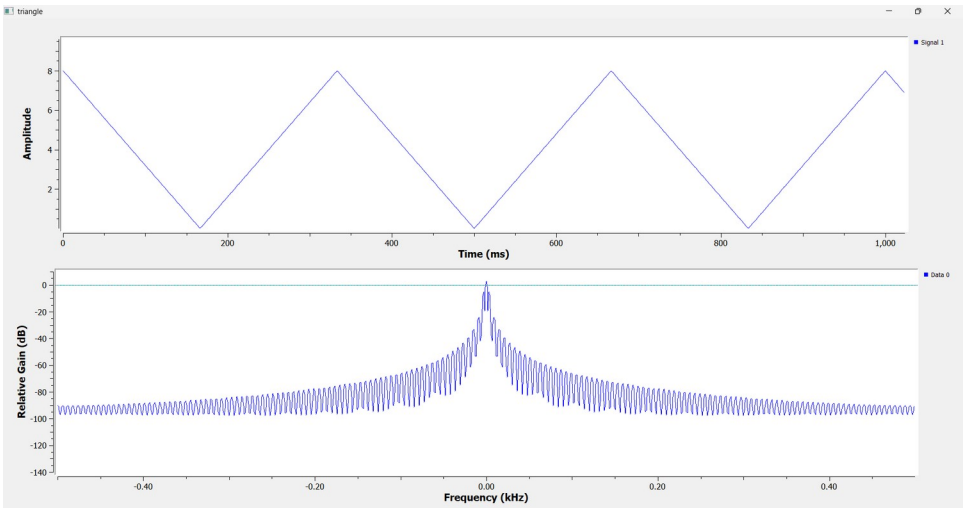
Time-domain (top) and frequency-domain (bottom) output — Cosine wave

6.1.3 Square Wave



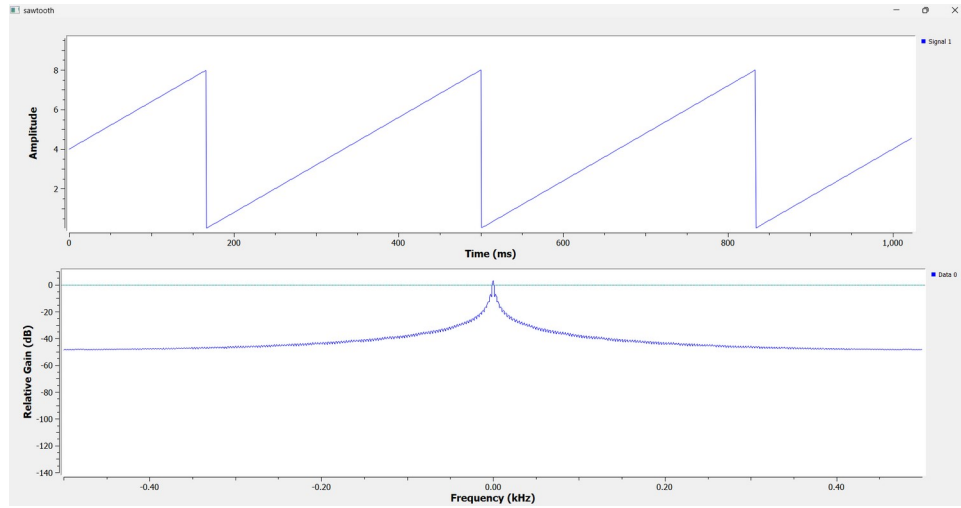
Time-domain (top) and frequency-domain (bottom) output — Square wave

6.1.4 Triangle Wave



Time-domain (top) and frequency-domain (bottom) output — Triangle wave

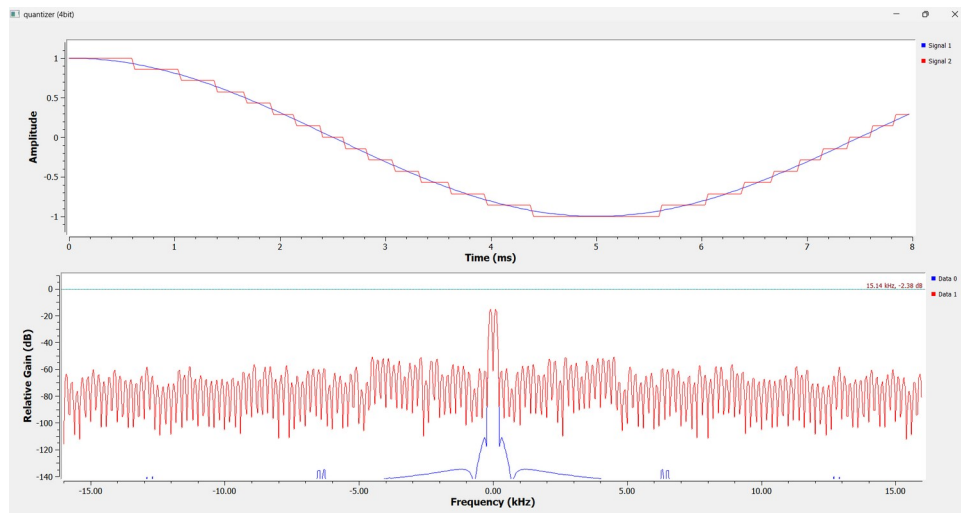
6.1.5 Sawtooth Wave



Time-domain (top) and frequency-domain (bottom) output — Sawtooth wave

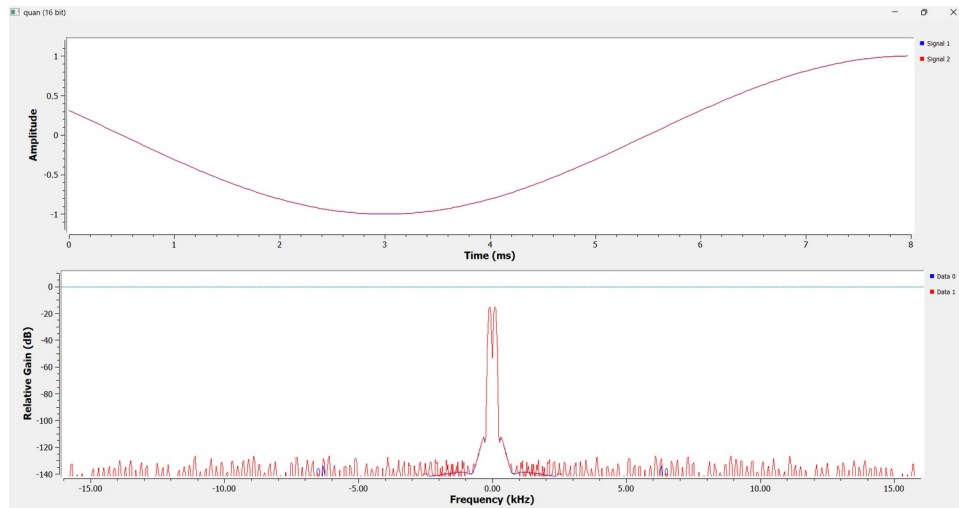
6.2 Quantization Outputs

6.2.1 4-bit Quantizer ($n = 4$)



Original cosine vs. 4-bit quantized cosine — time and frequency domain

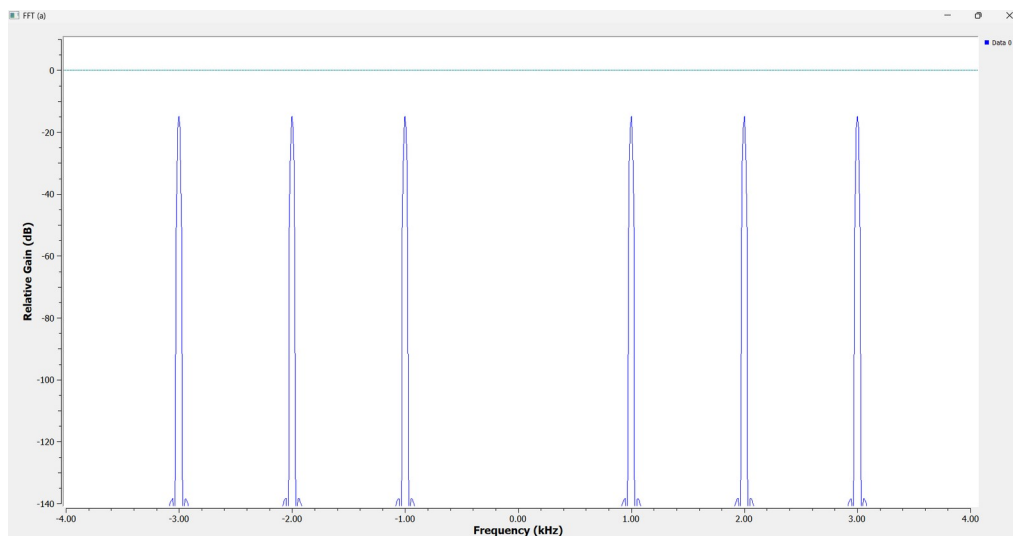
6.2.2 16-bit Quantizer ($n = 16$)



Original cosine vs. 16-bit quantized cosine — time and frequency domain

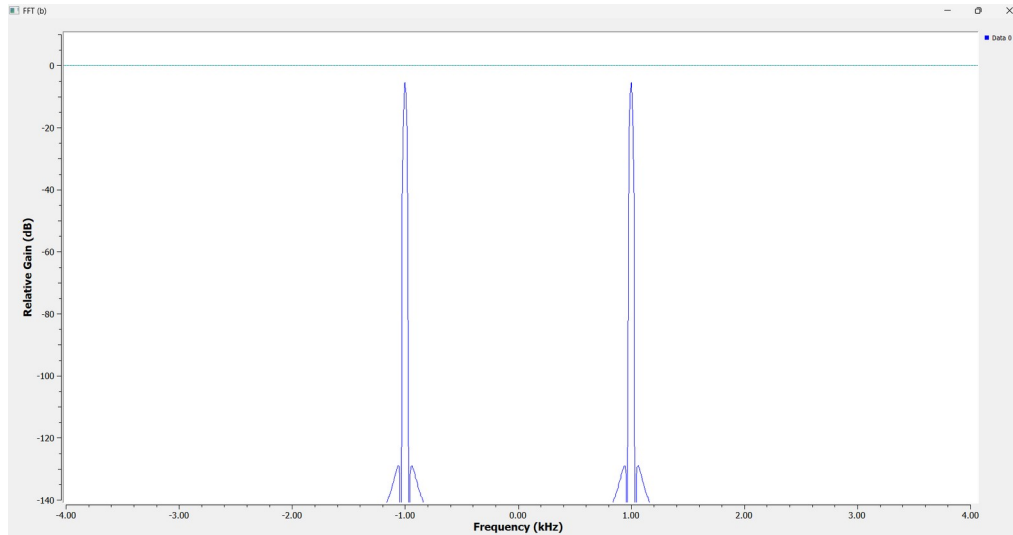
6.3 FFT / Aliasing Outputs

6.3.1 Case (a): $f_1 = 1$ kHz, $f_2 = 2$ kHz, $f_3 = 3$ kHz



FFT output — case (a): no aliasing

6.3.2 Case (b): $f_1 = 1$ kHz, $f_2 = 9$ kHz, $f_3 = 15$ kHz



FFT output — case (b): aliasing present

7. Result & Discussion

7.1 Waveform Generation

The Time Sink for the sine wave shows a smooth sinusoid oscillating between +8 and -8 with a period of about 333 ms (≈ 3 Hz), matching the configured frequency; its Frequency Sink shows a single sharp spectral line (plus symmetric image), confirming that a pure sine wave concentrates all its energy at one frequency. The cosine wave is identical in shape to the sine wave but phase-shifted by 90° , starting at its maximum value (+8) at $t = 0$; since a phase shift does not alter the magnitude spectrum, its frequency response is identical in shape and location to the sine wave's.

The square wave's Time Sink shows the expected two-level (0 to 8) periodic pulse train, and its Frequency Sink shows a dominant fundamental tone accompanied by a comb of odd harmonics, consistent with its Fourier series. The triangle wave shows the characteristic linear rise-and-fall shape; like the square wave it contains only odd harmonics, but because their amplitudes fall off faster ($1/n^2$ versus $1/n$), its harmonic sidelobes decay more quickly. The sawtooth wave shows the characteristic ramp-up/sharp-drop pattern, and because its Fourier series contains both even and odd harmonics with amplitude falling as $1/n$, its Frequency Sink shows the richest harmonic comb of the five waveforms, with a slower roll-off than the triangle wave but faster than the square wave.

7.2 Quantization

With only $n = 4$ bits (16 discrete levels across the ± 1 range), the quantized (red) trace follows the original cosine (blue) only approximately, forming a visible staircase pattern with large steps. In the frequency domain this coarse quantization manifests as a raised noise floor across the spectrum, with the fundamental tone sitting only about -2 dB relative to the reference level and the noise floor much closer to the signal peak — indicating a poor signal-to-quantization-noise ratio (SQNR).

With $n = 16$ bits (65536 discrete levels), the quantization step size becomes extremely small: the quantized and original traces are visually indistinguishable in the time domain, and in the frequency domain the quantized

signal's spectrum sits almost exactly on top of the original's, with the noise floor pushed down dramatically. This confirms the expected ~6 dB SQNR improvement per additional bit, giving roughly 70 dB better SQNR for the 16-bit quantizer over the 4-bit quantizer (12 extra bits).

7.3 FFT and Aliasing

In case (a), all three tones (1 kHz, 2 kHz, 3 kHz) lie well below the Nyquist frequency of $f_s/2 = 4$ kHz, so no aliasing occurs: the Frequency Sink shows six clean spectral peaks — three positive-frequency peaks and their conjugate-symmetric mirror images at negative frequencies — each appearing exactly where expected.

In case (b), $f_2 = 9$ kHz and $f_3 = 15$ kHz both exceed the Nyquist frequency of 4 kHz, so they are undersampled and alias back into the ± 4 kHz base band. Using $f_{alias} = |f - k \cdot f_s|$: for $f_2 = 9$ kHz, $9 - 8 = 1$ kHz; for $f_3 = 15$ kHz, $15 - 2 \times 8 = -1$ kHz $\rightarrow |-1| = 1$ kHz. All three input tones therefore collapse onto the same pair of spectral lines at ± 1 kHz, so only two peaks are visible instead of six. This directly demonstrates why the sampling rate must satisfy the sampling theorem ($f_s > 2 \times f_{max}$); otherwise higher-frequency components masquerade as false lower-frequency components in the sampled data, and this information loss cannot be undone after the fact.

8. Conclusion

This set of experiments, carried out in GNU Radio Companion, illustrated how the shape of a periodic waveform (sine, cosine, square, triangle, sawtooth) determines the richness of its harmonic content in the frequency domain. It further showed how the number of quantization bits directly controls the granularity of a digitized signal and its quantization-noise floor — 4-bit quantization introduces clearly visible distortion and a high noise floor, while 16-bit quantization is virtually indistinguishable from the original analog signal. Finally, it demonstrated how respecting or violating the Nyquist sampling criterion determines whether an FFT correctly represents a signal's true frequency content or produces misleading aliased components. Together these results reinforce the practical importance of the sampling theorem and anti-aliasing filtering in any real-world analog-to-digital signal processing chain.